Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Кафедра инженерной психологии и эргономики

Специальность «Программная инженерия»

Учебная дисциплина «Компьютерные системы и сети»

ОТЧЕТ

по лабораторной работе №2
«Traceroute»

Подготовил: Козинцев М.Л.

Проверил: Болтак С. В.

Минск 2026

Задание: разработать простейший аналог утилиты tracert, с возможностью задавать имя целевого узла вместо его IP-адреса. Добавить параметр командной строки, активирующий режим разрешения в имена узлов через обратные DNS-запросы.

В рамках данной работы будет использован протокол ICMP. Из-за блокировки сетевым экраном ICMP-пакетов с типом «TTL expired» на время работы программы сетевой экран должен быть отключен.

Для запуска программы необходимо перейти к местоположению исполняемого файла в командной строке с правами администратора и ввести команду “mytracert [-r] <IP-адрес или имя целевого узла>”. В команде можно поставить параметр «-r», который включает режим разрешения в имена узлов. В рамках текущей работы для трассировки определены следующие параметры: отправляется 4 пакета, максимальное время ожидания ответа – 3 секунды, максимальное количество «прыжков» - 40.

На рисунке 1 представлен результат работы программы без параметра «-r».

```
D:\Uni\KSiS\KSiS_LR_Kozintsev\Lab1\MyTracert>mytracert bsuir.by
Трассировка маршрута к bsuir.by [134.17.213.43]
с максимальным числом прыжков 40:

  1     2 мс    1 мс    2 мс    1 мс  192.168.0.1
  2     *      10 мс    6 мс    *     10.5.0.1
  3     6 мс    6 мс    7 мс    7 мс  10.254.95.185
  4    10 мс    4 мс    5 мс    8 мс  195.137.180.26
  5     4 мс    4 мс    4 мс    3 мс  172.24.145.5
  6     4 мс    3 мс    3 мс    3 мс  172.24.145.6
  7     6 мс    4 мс    5 мс    7 мс  172.24.145.1
  8     3 мс    4 мс    3 мс    5 мс  134.17.208.1
  9     4 мс    3 мс    3 мс    3 мс  134.17.210.100
 10     4 мс    7 мс    4 мс    5 мс  134.17.213.43
 11     4 мс    5 мс    5 мс    5 мс  134.17.213.43

Трассировка завершена.
```

Рисунок 1 – Трассировка маршрута до узла bsuir.by без разрешения в имена узлов

На рисунке 2 представлен результат работы программы с параметром «-r».

```
D:\Uni\KSiS\KSiS_LR_Kozintsev\Lab1\MyTracert>mytracert -r bsuir.by
Трассировка маршрута к bsuir.by [134.17.213.43]
с максимальным числом прыжков 40:

  1     2 мс    2 мс    1 мс    1 мс  192.168.0.1
  2     7 мс    *       9 мс    9 мс  10.5.0.1
  3    10 мс   12 мс    7 мс   11 мс  10.254.95.185
  4     4 мс    4 мс    4 мс    4 мс  195.137.180.26
  5     4 мс    5 мс    3 мс    5 мс  172.24.145.5
  6     4 мс    3 мс    3 мс    3 мс  172.24.145.6
  7     8 мс    4 мс    4 мс    4 мс  172.24.145.1
  8     5 мс    4 мс    5 мс    5 мс  134.17.208.1
  9     4 мс    3 мс    3 мс    5 мс  100-210-17-134-dynamic-pool.internet.mts.by [134.17.210.100]
 10     5 мс    5 мс    4 мс    6 мс  mail.bsuir.by [134.17.213.43]
 11     6 мс    8 мс    5 мс    8 мс  mail.bsuir.by [134.17.213.43]

Трассировка завершена.
```

Рисунок 2 – Трассировка маршрута до узла bsuir.by с разрешением в имена узлов

На рисунке 3 представлен аналогичный запрос, выполненный утилитой tracert.

```
D:\Uni\KSiS\KSiS_LR_Kozintsev\Lab1\MyTracert>tracert bsuir.by

Трассировка маршрута к bsuir.by [134.17.213.43]
с максимальным числом прыжков 30:

  1     2 ms     1 ms     1 ms  192.168.0.1
  2     *        *        8 ms  10.5.0.1
  3     5 ms     4 ms     4 ms  10.254.95.185
  4    42 ms     7 ms     3 ms  195.137.180.26
  5     3 ms     4 ms     4 ms  172.24.145.5
  6     3 ms     2 ms     3 ms  172.24.145.6
  7     5 ms     3 ms     3 ms  172.24.145.1
  8     4 ms     5 ms     3 ms  134.17.208.1
  9     4 ms     4 ms     5 ms  100-210-17-134-dynamic-pool.internet.mts.by [134.17.210.100]
 10     5 ms     5 ms     4 ms  mail.bsuir.by [134.17.213.43]
 11     7 ms     5 ms     4 ms  mail.bsuir.by [134.17.213.43]

Трассировка завершена.
```

Рисунок 3 – Трассировка маршрута до узла bsuir.by утилитой tracert

Анализируя рисунки 1-3 видно, что пакеты утилиты tracert и программы mytracert проходят один маршрут.

На рисунке 4 представлен фрагмент захвата трафика с фильтром «icmp», генерируемого программой.

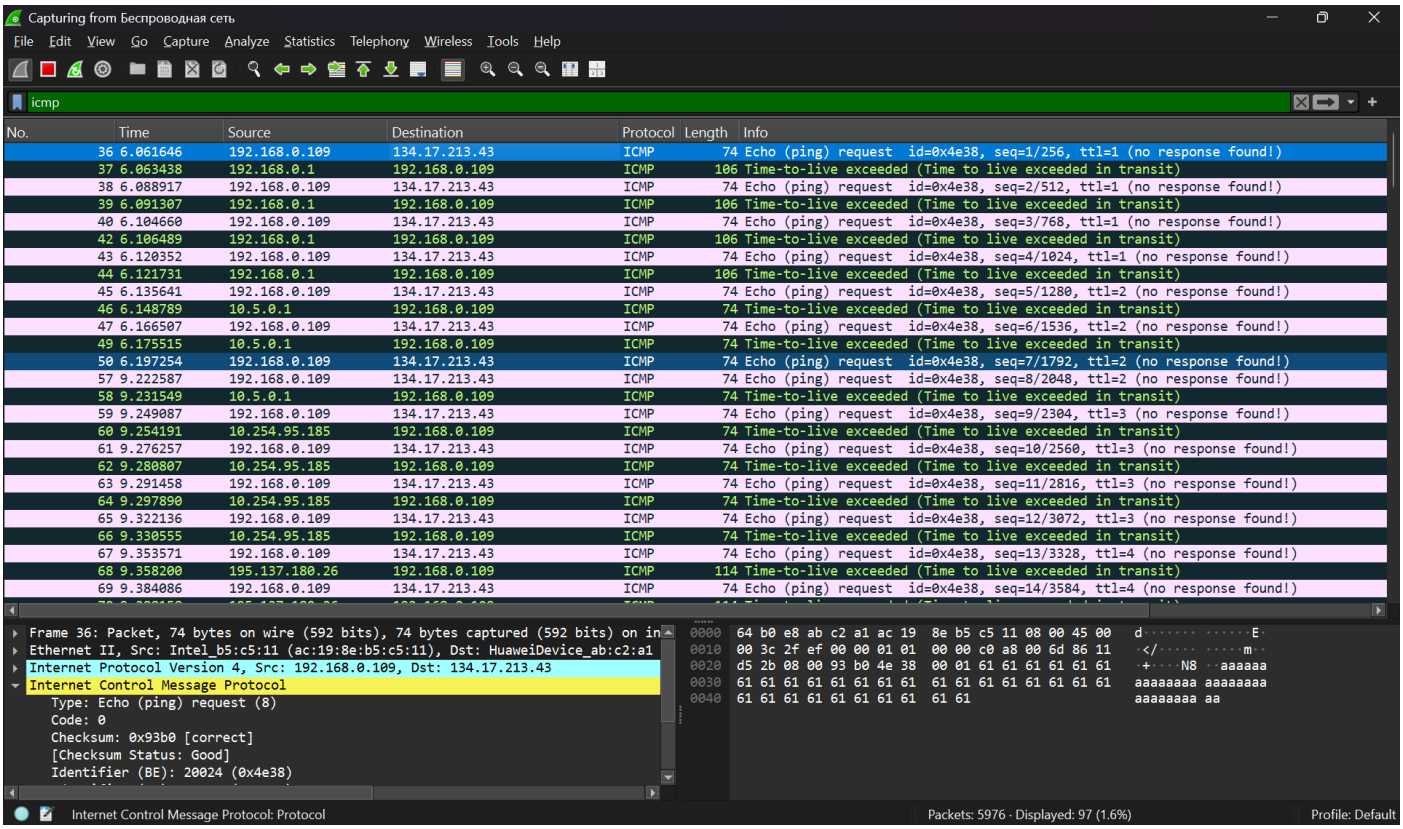

Рисунок 4 – Захват трафика в Wireshark

На рисунке 5 представлена информация об одном из перехваченных ICMP пакетов.

```
▸ Frame 36: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on int
▸ Ethernet II, Src: Intel_b5:c5:11 (ac:19:8e:b5:c5:11), Dst: HuaweiDevice_ab:c2:a1 (
▸ Internet Protocol Version 4, Src: 192.168.0.109, Dst: 134.17.213.43
▾ Internet Control Message Protocol
    Type: Echo (ping) request (8)
    Code: 0
    Checksum: 0x93b0 [correct]
    [Checksum Status: Good]
    Identifier (BE): 20024 (0x4e38)
    Identifier (LE): 14414 (0x384e)
    Sequence Number (BE): 1 (0x0001)
    Sequence Number (LE): 256 (0x0100)
  ▸ [No response seen]
  ▸ Data (32 bytes)
```

Рисунок 5 – Информация об ICMP запросе (корректный формат Sequence number и Identifier)

Вывод: в ходе выполнения лабораторной работы был разработан аналог утилиты tracert с использованием сырых сокетов и протокола ICMP. Результаты работы программы идентичны результату работы системной утилиты tracert. Перехваченный трафик, полученный из сниффера Wireshark, также подтверждает корректную работу программы. Дополнительно был реализован режим разрешения в имена узлов через обратные DNS-запросы, активируемый параметром «-r» командной строки.

Листинг кода программы:

```
using System;
using System.Diagnostics;
using System.Net;
using System.Net.Sockets;
using System.Threading;

namespace MyTracert
{
    class Program
    {
        //Константы для ICMP
        private const int IcmpEchoRequestType = 8;
```

```
private const int IcmpEchoReplyType = 0;
private const int IcmpTimeExceededType = 11;

private const int MaxHops = 40;
private const int TimeoutMs = 3000;
private const int PacketsPerHop = 4;

static void Main(string[] args)
{
    if (args.Length == 0)
    {
        Console.WriteLine("Использование: mytracert [-r] <IP или Имя узла>");
        Console.WriteLine("  -r : Переводить IP-адреса в имена узлов");
        return;
    }

    bool resolveNames = false;
    string targetHost = "";

    //Парсинг аргументов
    foreach (var arg in args)
    {
        if (arg == "-r") resolveNames = true;
        else targetHost = arg;
    }

    if (string.IsNullOrEmpty(targetHost))
    {
        Console.WriteLine("Не указан целевой узел.");
        return;
```

```
            }

            IPAddress targetIp;
            try
            {
                //Попытка перевода доменного имени в IP
                IPAddress[] addresses = Dns.GetHostAddresses(targetHost);
                targetIp = addresses[0];
            }
            catch (Exception)
            {
                Console.WriteLine($"Не удается разрешить системное имя узла {targetHost}.");
                return;
            }

            Console.WriteLine($"Трассировка маршрута к {targetHost} [{targetIp}]");
            Console.WriteLine($"с максимальным числом прыжков {MaxHops}:\n");

            //Сырой сокет для ICMP
            using Socket icmpSocket = new Socket(AddressFamily.InterNetwork, SocketType.Raw, ProtocolType.Icmp);
            icmpSocket.ReceiveTimeout = TimeoutMs;
            icmpSocket.SendTimeout = TimeoutMs;

            ushort sequenceNumber = 1;
            ushort processId = (ushort)Process.GetCurrentProcess().Id;

            bool reachedDestination = false;

            for (int ttl = 1; ttl <= MaxHops && !reachedDestination; ttl++)
```

```
{
    //Установка TTL для текущего прыжка
    icmpSocket.SetSocketOption(SocketOptionLevel.IP,
SocketOptionName.IpTimeToLive, ttl);

    Console.Write($"{ttl,3} ");

    IPAddress replyAddress = null;

    for (int i = 0; i < PacketsPerHop; i++)
    {
        //Создание пакета и обновление sequence number
        byte[] packet = CreateIcmpEchoRequest(processId, sequenceNumber);
        sequenceNumber++;



        EndPoint remoteEndPoint = new IPEndPoint(targetIp, 0);
        Stopwatch timer = new Stopwatch();

        try
        {
            timer.Start();
            icmpSocket.SendTo(packet, remoteEndPoint);

            //Буфер для получения ответа
            byte[] receiveBuffer = new byte[256];
            EndPoint replyEndPoint = new IPEndPoint(IPAddress.Any, 0);

            //Ожидание ответа
            int bytesRead = icmpSocket.ReceiveFrom(receiveBuffer, ref
replyEndPoint);
```

```
            timer.Stop();

            replyAddress = ((IPEndPoint)replyEndPoint).Address;

            //Время ответа
            if (timer.ElapsedMilliseconds == 0)
                Console.Write("    < 1 мс");
            else
                Console.Write($"{timer.ElapsedMilliseconds,5} мс");

            //Анализ полученного ICMP пакета (пропуск заголовка)
            int ipHeaderLength = (receiveBuffer[0] & 0x0F) * 4;
            int icmpType = receiveBuffer[ipHeaderLength];

            if (icmpType == IcmpEchoReplyType)
            {
                reachedDestination = true;
            }
        }
        catch (SocketException)
        {
            //Обработка таймаута ожидания
            Console.Write("\t*   ");
        }

        Thread.Sleep(10);
    }

    //Вывод адреса и имени узла
    if (replyAddress != null)
```

```
        {
            string hostNameStr = replyAddress.ToString();
            if (resolveNames)
            {
                try
                {
                    IPHostEntry hostEntry = Dns.GetHostEntry(replyAddress);
                    hostNameStr = $"{hostEntry.HostName} [{replyAddress}]";
                }
                catch { }
            }
            Console.Write($"  {hostNameStr}");
        }
        else
        {
            Console.Write("  Превышен интервал ожидания для запроса.");
        }

        Console.WriteLine();
    }

    Console.WriteLine("\nТрассировка завершена.");
}

//Метод для создания массива байтов для ICMP запроса
private static byte[] CreateIcmpEchoRequest(ushort id, ushort sequence)
{
    //Заголовок ICMP - 8 байт, полезная нагрузка - 32 байта
    int packetSize = 8 + 32;
    byte[] packet = new byte[packetSize];
```

```
packet[0] = IcmpEchoRequestType; //Type 8 (Echo Request)
packet[1] = 0;                   //Code 0

//Checksum - байты 2 и 3
//ПЕРЕРАБОТАННЫЕ ВЫЧИСЛЕНИЯ
//Identifier
short networkId = IPAddress.HostToNetworkOrder((short)id);
byte[] idBytes = BitConverter.GetBytes(networkId);
packet[4] = idBytes[0];
packet[5] = idBytes[1];

//Sequence Number
short networkSequence = IPAddress.HostToNetworkOrder((short)sequence);
byte[] seqBytes = BitConverter.GetBytes(networkSequence);
packet[6] = seqBytes[0];
packet[7] = seqBytes[1];
//Заполнение полезной нагрузки
for (int i = 8; i < packetSize; i++)
{
    packet[i] = (byte)'a';
}

//Контрольная сумма
ushort checksum = CalculateChecksum(packet);
byte[] checksumBytes = BitConverter.GetBytes(checksum);
packet[2] = checksumBytes[0];
packet[3] = checksumBytes[1];

return packet;
```

```
        }

        //Метод для вычисления контрольной суммы
        private static ushort CalculateChecksum(byte[] buffer)
        {
            int checksum = 0;
            int length = buffer.Length;
            int index = 0;

            while (length > 1)
            {
                checksum += Convert.ToInt32(BitConverter.ToUInt16(buffer, index));
                index += 2;
                length -= 2;
            }

            if (length > 0)
            {
                checksum += Convert.ToInt32(buffer[index]);
            }

            checksum = (checksum >> 16) + (checksum & 0xffff);
            checksum += (checksum >> 16);

            return (ushort)(~checksum);
        }
    }
}
```